2.1、图像基操

https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html）这篇其实教的是「用 NumPy 的方式操作 OpenCV 图像」，而不是大量调用 cv 函数。
掌握它能让你写出更快、更灵活的代码。

1. 读取图片（基本起手式）

```
import cv2 as cv
import numpy as np

img = cv.imread('messi5.jpg')
if img is None:
    print("图片没读到！检查路径")
    exit()
```

2. 访问 & 修改单个像素（不推荐大量使用）

```
# 获取 (x=100, y=100) 位置的像素值（注意顺序：行、列）
px = img[100, 100]          # → [B, G, R] 例如 [157 166 200]
print(px)

# 只拿蓝色通道
blue = img[100, 100, 0]     # → 157
print(blue)

# 修改像素为白色
img[100, 100] = [255, 255, 255]
```

重要警告（官方三次强调）
千万不要循环一个一个像素去读写！非常非常慢！
应该尽量使用：

- 整块赋值
- 掩码（mask）
- 矢量化运算（+ - * /）
- cv 函数

3. 图像的三个最常用属性

```
print(img.shape)   # (高度, 宽度, 通道数) → 彩色图是三维 (512, 512, 3)
print(img.size)    # 总像素个数 = 高×宽×通道
print(img.dtype)   # 几乎永远是 uint8，错了基本就出问题
```

调试时先 print 这三个，能解决 50% 的奇怪 bug。

4. 抠图 & 贴图（ROI — Region of Interest）最经典的操作之一：

OpenCV（以及几乎所有主流图像处理库、matplotlib、PIL等）的图像坐标系都是：

- 原点 (0,0) 在左上角
- x 向右递增 → 对应宽度（width / 列数）
- y 向下递增 → 对应高度（height / 行数）
- 并且 `img[y1:y2,x1:x2]`，是先写y再写x

```
# 假设我们抠 messi 的球（坐标自己看图量）
ball = img[280:340, 330:390]     # y从280到340，x从330到390

# 贴到图片左边一点点的位置
img[273:333, 100:160] = ball
```

这就是最简单、最快的「复制粘贴」方式。

5. 通道拆分与合并（两种写法对比）

写法1：用 cv 函数（比较慢）

```
b, g, r = cv.split(img)
img = cv.merge([b, g, r])   # 恢复原图
```

写法2：用 NumPy 切片（推荐，速度快很多）

```
b = img[:, :, 0]
g = img[:, :, 1]
r = img[:, :, 2]

# 直接把红色通道全部清零（变成蓝绿色调）
img[:, :, 2] = 0
```

结论：除非你真的需要把三个通道分开成三个独立数组，否则优先用 img[:,:,c]。

6. 给图像加边框（padding） — cv.copyMakeBorder

这是很多卷积、深度学习预处理、展示结果时都会用到的功能。常用几种 borderType：

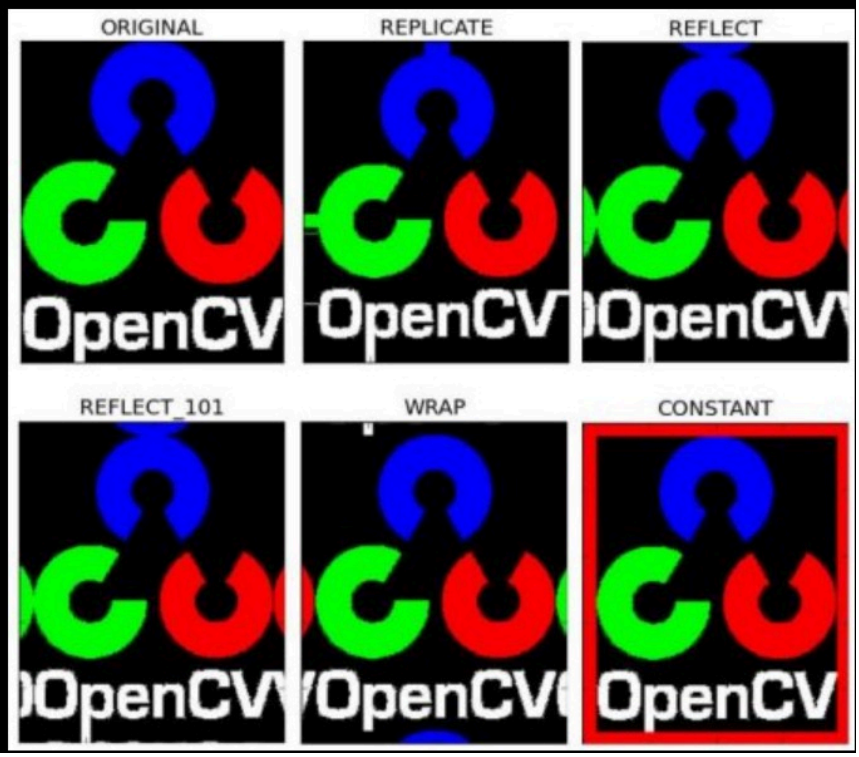
写法	效果说明	常见用途
BORDER_CONSTANT	固定颜色（通常黑色/白色/自定义）	最常用
BORDER_REPLICATE	边缘像素无限复制	平滑边缘
BORDER_REFLECT	镜面反射（不包括最边像素）	对称性好
BORDER_REFLECT_101	OpenCV 默认，反射（包含最边像素）	卷积/滤波最常用
BORDER_WRAP	像俄罗斯方块一样循环	很少用

代码示例（建议自己跑一下看效果）：

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img = cv.imread('opencv-logo.png')

blue = [255, 0, 0]  # BGR 顺序！

replicate  = cv.copyMakeBorder(img, 20,20,20,20, cv.BORDER_REPLICATE)
reflect    = cv.copyMakeBorder(img, 20,20,20,20, cv.BORDER_REFLECT)
```

```
reflect101 = cv.copyMakeBorder(img, 20,20,20,20, cv.BORDER_REFLECT_101)
wrap       = cv.copyMakeBorder(img, 20,20,20,20, cv.BORDER_WRAP)
const      = cv.copyMakeBorder(img, 20,20,20,20, cv.BORDER_CONSTANT,
value=blue)

titles = ['原图', 'REPLICATE', 'REFLECT', 'REFLECT_101', 'WRAP', 'CONSTANT']
images = [img, replicate, reflect, reflect101, wrap, const]

for i in range(6):
    plt.subplot(2, 3, i+1)
    plt.imshow(cv.cvtColor(images[i], cv.COLOR_BGR2RGB))  # matplotlib用RGB
    plt.title(titles[i])
    plt.axis('off')
plt.show()
```

请看下面的结果。(图片显示时使用matplotlib。因此，红色和蓝色通道将互换)：



当你用 matplotlib.pyplot.imshow() 直接显示 OpenCV 读取的 BGR 图像时，会出现颜色失真：

- OpenCV 内部存储顺序：BGR（蓝-绿-红）
- matplotlib / 大多数显示库期望：RGB（红-绿-蓝）
  所以不做转换直接 plt.imshow(img) 会导致：红色和蓝色通道互换

快速总结 - 核心技巧口诀

- 读属性：shape / size / dtype 先看
- 改像素：能整块改就不要for循环

- 抠图贴图：直接用切片 [y1:y2, x1:x2]
- 通道操作：优先 img[:,:,0/1/2]，少用 split/merge
- 加padding：copyMakeBorder + BORDER_REFLECT_101 最常用
- 记住 BGR 顺序（和大家习惯的 RGB 相反）